ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name:**
- **Chinese Name: 陳熙緁**
- **Email: s13150247@thu.edu.tw**
- **Student ID: s13150247**
- **University: THU**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**

    - Copying assignment questions directly into an AI tool to generate answers.
    - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
    - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. There are 594 documents in the category sci.med.
2. Isolating specific categories is a crucial first step before applying algorithms like frequent pattern mining for three reasons.

   First, it helps us find the hidden pattern or trend better. Patterns and trends are more obvious in one specific category, especially when

patterns or trends only appear in the specific category. Some words only appear in the specific category, so the hidden information is easier to recognize while dealing with one category. If we don't isolate specific categories, the denominator will become bigger and the proportion will be diluted. When a pattern's support doesn't reach the minimum support threshold, it may be filtered out.

Second, other categories will interfere. One feature may appear in different categories, but the meaning may be totally different or may have no special meaning, causing valuable information to be buried. For example, the word "think" appears in both religion and medicine, but it means nothing special, so it may hide other information that needs to be found.

Finally, it can reduce the computational cost. More data requires more time, money, and computing power to process. So, dealing with the specific categories can reduce the cost.

AI used for this answer: Yes
If yes, how: Concept explanation (how minimum support works in frequent pattern mining), feedback on the logic of my own draft through guiding questions, and grammar correction of my self-written English answer.
External references (links):

- **External references (links):** PAMI FPGrowth documentation: https://colab.research.google.com/github/UdayLab/PAMI/blob/main/notebooks/frequentPattern/basic/FPGrowth.ipynb
- Tan, Steinbach & Kumar, *Introduction to Data Mining*, Association Analysis chapter (definition of support)

Question 2: Handling Missing Data
Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)
Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.
Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.
Answer:

Though 40% of the documents are missing 'category_name', we can still tell the category by 'category'. If we just delete those documents, we will lose a large amount of usable data. At the same time, if the missing data are concentrated in certain categories, deleting them may cause bias. So, we can use the remaining 60% of the documents to find out which 'category_name' each 'category' maps to. Because the mapping is one-to-one, no guessing is needed. By filling in the

missing labels, the 40% of the documents become usable again, so their text data can still be utilized. The method is called Estimate Missing Values.

AI used for this answer: Yes
If yes, how: Concept explanation (why ignoring missing values does not fit when the missing value is the class label, and how the four methods for handling missing values differ), feedback on the logic of my own draft through guiding questions, and grammar correction of my self-written English answer.
External references (links):

- pandas documentation, Working with missing data: https://pandas.pydata.org/docs/user_guide/missing_data.html
- Tan, Steinbach & Kumar, *Introduction to Data Mining*, Data chapter, section on data quality (handling missing values)

Question 3: Data Sampling Risks
Notebook: Section 5.1 — Question 3 (take home)
Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.
Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. No, they don't. The ranking of the categories changed. For example, comp.graphics made up 25.9% of the original data, but 27.8% of the sample.
2. The risk is that simple random sampling chooses documents purely by chance, so when the dataset is highly imbalanced, the categories with fewer documents may be underrepresented or even completely missing, and it may cause bias.

AI used for this answer: Yes
If yes, how: Concept explanation (the risk of simple random sampling on imbalanced data), guidance on reading the correct output (comparing the printed category counts instead of the bar chart), and grammar correction of my self-written English answer.

- **External references (links):** pandas documentation, DataFrame.sample: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sample.html
- Tan, Steinbach & Kumar, *Introduction to Data Mining*, Data chapter, section on sampling (simple random sampling and stratified sampling)

Question 4: Text Vectorization and Matrix Sparsity
Notebook: Section 5.3 — Question 4 (take home)
Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. The total number of columns increased from 35482 to 35788. The 306 extra columns are the stopwords that were added back.
2. The overall sparsity of the matrix decreased from 99.6772% to 99.5470%. It's because the stopwords appear in almost every document, so their columns are mostly non-zero. So when we add the stopwords back, the percentage of 0 decreased. However, only 306 columns were added out of more than 35,000, so the percentage did not decrease so much.

AI used for this answer: Yes
If yes, how: Concept explanation (what a document-term matrix and sparsity are), a suggested line of code to calculate sparsity so I could verify the result myself, help fixing my Jupyter kernel and working-directory errors, and grammar correction of my self-written English answer.
External references (links):

- scikit-learndocumentation,CountVectorizer:https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

---

Question 5: Feature Selection — Variance Thresholding
Notebook: Section 5.3.1 — Question 5 (take home)
Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.
Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	Number of terms kept (keep == True)
0.0	35482
0.01	5089
0.05	1491

1. The total size of the vocabulary decreased from 35,482 to 5,089, and then to 1,491. Notably, when we only raised the threshold from 0 to 0.01, almost 85% of the words were removed, which means most words in this matrix have low variance.
2. Words with variance close to zero are words that appear in very few documents, such as typos, rare proper nouns, or unusual strings.
3. Near-zero variance features do not appear in many documents, so even if we delete them, it won't affect the model much. Since we use features to distinguish documents, features that barely change across documents carry almost no useful information, so removing them hardly affects the result.

AI used for this answer: Yes
If yes, how: Concept explanation (what variance means for a word column), help fixing a variable-mismatch error in my notebook, and grammar correction of my self-written English answer.
External references (links):
scikit-learn documentation, VarianceThreshold: https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.VarianceThreshold.html

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)
Notebook: Section 5.3.2 — Question 6 (take home)
Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.
Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. The top-correlated words are domain-specific terminology for graphics, such as "graphics", "files", and "images".
2. Pearson correlation measures the linear relationship between the raw values, which means how well the data points fit a straight line. Spearman

rank correlation first converts the values into ranks, and measures the monotonic relationship, which means whether one value consistently increases as the other increases, even if the relationship is not a straight line.

3. When a word appears in almost every document of the topic, but one or a few documents use it extremely many times while the others use it only once or twice, the word would have a high Spearman rank correlation but a significantly lower Pearson correlation. This is because the extreme counts act as outliers that pull down Pearson, while Spearman only looks at ranks, so the outliers barely affect it.

AI used for this answer: Yes
If yes, how: Concept explanation (linear vs. monotonic correlation, with a toy example showing how outliers affect Pearson but not Spearman), and grammar correction of my self-written English answer.
External references (links):
SciPy documentation, spearmanr:
https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics
Notebook: Section 5.4.2 — Question 7 (take home)
Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.
Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:
1.When minSup is lowered from 9 to 3, both the runtime and the total number of patterns increase. The number of patterns rises from 710 to 9,547, and the runtime increases from about 0.07 seconds to about 1.36 seconds. This is because a lower threshold lets more word combinations pass, so the algorithm has to search and count many more combinations, which takes more time.
Answer (2) — Top-K k vs MaxFPGrowth minSup:
2. Top-K fixes the number of patterns (k), but the minimum number of times a kept pattern must appear changes depending on the data. minSup fixes the minimum number of occurrences, but the final number of patterns found changes depending on the data.
Answer (3) — Maximal vs standard frequent patterns:
3.Maximal frequent patterns only keep the largest frequent patterns, which would no longer be frequent if any one more word were added. Therefore, there are far fewer maximal frequent patterns than standard frequent patterns. The smaller combinations that are left out are all subsets of a maximal pattern, so they must

also be frequent and can be recovered from it. As a result, maximal frequent patterns remove a lot of redundancy without losing the information about which combinations are frequent.

AI used for this answer: Yes
If yes, how: Concept explanation (what frequent and maximal frequent patterns are, using a shopping-receipt example), feedback on the logic of my draft through guiding questions, suggested wording for part 2, and translation of my Chinese answer into English.
External references (links):

- PAMI MaxFPGrowth notebook: https://colab.research.google.com/github/UdayLab/PAMI/blob/main/notebooks/frequentPattern/maximal/MaxFPGrowth.ipynb
- PAMI FAE (Top-K) notebook: https://colab.research.google.com/github/UdayLab/PAMI/blob/main/notebooks/frequentPattern/topk/FAE.ipynb

---

Question 8: Dimensionality Reduction — Motivation
Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)
Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).
Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:
1. The original matrix has far more than three dimensions, so it cannot be shown as raw data, either mathematically or visually. X_counts has 35,482 columns, which means each document is a point with 35,482 coordinates (35,482 dimensions). A scatter plot can only have two axes, and even a 3D plot can only have three, so these points cannot be drawn on a 2D or 3D plot.
2. Dimensionality reduction techniques, such as Principal Component Analysis (PCA), t-Distributed Stochastic Neighbor Embedding (t-SNE), and Uniform Manifold Approximation and Projection (UMAP). They compress the high-dimensional data into two or three dimensions so that it can be plotted.
AI used for this answer: Yes
If yes, how: Concept explanation (what the dimensions of the document-term matrix mean and why they cannot be plotted directly), hints toward the correct algorithm category, and translation of my Chinese answer into English.
External references (links):

- scikit-learn documentation, PCA: https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. UMAP shows the four categories most clearly as separated clusters, although some of them are still partly mixed. PCA shows the heaviest overlap around the center.
2. PCA works with linear relationships. It keeps the directions with the largest overall variance, but high-dimensional text data usually has non-linear structure. t-SNE and UMAP are designed for non-linear relationships and keep the relationship between each point and its nearby points, so they are better at separating the categories.
3. t-SNE keeps the relationship between each point and its nearby points, but it does not preserve the distances between separate clusters. Also, t-SNE is very sensitive to its parameters, so a large distance on the plot does not necessarily mean a large real difference.

AI used for this answer: Yes

If yes, how: Concept explanation (what PCA and t-SNE/UMAP preserve), feedback on the logic of my draft through guiding questions, and translation of my Chinese answer into English.

External references (links):

scikit-learn documentation, TSNE: https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html

---

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. 4 binary columns are created, one for each category.

2. If the categories are represented as 0, 1, 2, and 3, the algorithm will wrongly assume that the categories have different sizes. This makes it believe there is an order (for example, 3 > 2 > 1 > 0) and a distance between them. For example, category 0 would seem closer to category 1 than to category 3, which can mislead distance-based algorithms such as KNN.

3. With one-hot binarization, each category gets its own column with only 0 or 1, which shows only whether a document belongs to that category, without any size. This removes the false order, and the distance between any two categories becomes the same.

AI used for this answer: Yes
If yes, how: Concept explanation (what binary columns are and why integer labels imply order and distance), feedback on my draft through guiding questions, help recreating variables after restarting the notebook, and translation of my Chinese answer into English.
External references (links):
scikit-learn documentation, LabelBinarizer: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelBinarizer.html

---